3. Bubble Sort with Big-O Analysis

Aim: To sort an array using Bubble Sort and analyze its time complexity using Big-O notation.

Algorithm

1. Start the program.
2. Read the size of the array.
3. Read the array elements.
4. Compare each pair of adjacent elements.
5. If the first element is greater than the second, swap them.
6. Repeat the process until the array is sorted.
7. Display the sorted array.
8. Stop the program.

C

```
#include <stdio.h>

int main() {
    int a[100], n, i, j, temp;
    printf("Enter the size of array: ");
    scanf("%d", &n);
    printf("Enter the array elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
```

```
    printf("Sorted array: ");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

Input: Enter array elements: 64 34 25 12 22 11 90

Output:

```
Enter the size of array: 7
Enter the array elements: 64 34 25 12 22 11 90
Sorted array: 11 12 22 25 34 64 90
```

Result: Thus, the C program successfully sorts the array in ascending order using Bubble Sort and analyzes its time complexity using Big-O notation.